Lab 12: DevOps, Docker, Kubernetes & CI/CD

Z5008 Big Data Lab

Dr. Innocent Nyalala
innocent@iitmz.ac.in

School of Engineering and Science
IIT Madras Zanzibar

Monday, 1 June 2026

"Ship code, not excuses." — DevOps maxim

Agenda

Why DevOps for Big Data?

The old way

- "Works on my machine"
- Manual server setup weeks before launch
- Snowflake servers—undocumented configs
- No rollback strategy
- Data scientists hand off to separate Ops team

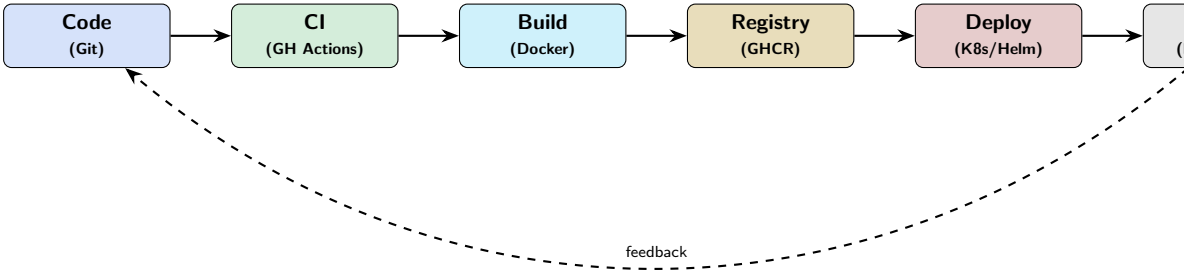
The DevOps way

- Immutable, reproducible containers
- Infrastructure as Code (IaC)
- Automated pipelines: lint $\rightarrow$ test $\rightarrow$ build $\rightarrow$ deploy
- Kubernetes scales Spark executors automatically
- One team owns the full lifecycle

Lab 12 Goal

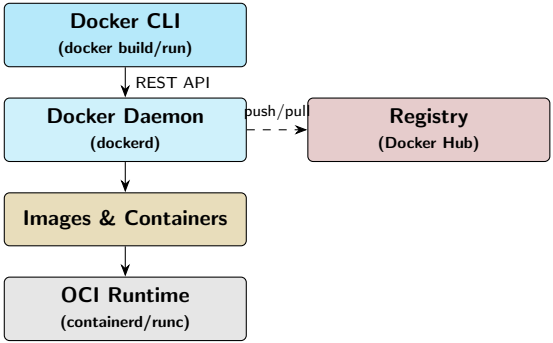
Containerise the PySpark pipeline built in Labs 1–11, orchestrate it on Kubernetes, and automate delivery via GitHub Actions.

DevOps Toolchain Overview



- **Git** → triggers **CI** → builds **Docker image** → pushes to **registry**
- **K8s/Helm** pulls image and creates/updates workloads
- Metrics feed back into development decisions

Docker Architecture



Key concepts

- **Image**: read-only layered FS snapshot
- **Container**: running instance (writable layer on top)
- **Layer caching**: unchanged layers reused on rebuild
- **Registry**: remote store for images (Hub, GHCR, ECR, GCR)
- **Daemon**: background service managing containers

Dockerfile: Multi-Stage Build for PySpark

```
# syntax=docker/dockerfile:1
# ---- Stage 1: builder ----
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---- Stage 2: runtime ----
FROM eclipse-temurin:17-jre-jammy AS runtime
ARG SPARK_VERSION=3.5.1
ENV SPARK_HOME=/opt/spark \
    PYSPARK_PYTHON=/usr/local/bin/python3 \
    PATH="/opt/spark/bin:$PATH"

RUN apt-get update && apt-get install -y --no-install-recommends \
        python3 python3-pip curl \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /install /usr/local

RUN curl -fSL https://archive.apache.org/dist/spark/spark-${SPARK_VERSION}/\
spark-${SPARK_VERSION}-bin-hadoop3.tgz | tar -xz -C /opt/ \
```

Dockerfile Best Practices

.dockerignore

```
__pycache__/
*.pyc
.git/
.pytest_cache/
*.egg-info/
data/raw/
notebooks/
```

Layer-cache tricks

- Copy `requirements.txt` *before* source code
- Chain `RUN` with `&&` to reduce layers
- Use `--no-cache-dir` for pip
- Prefer `COPY` over `ADD`

ARG vs ENV

- `ARG`: build-time variable, not in final image
- `ENV`: runtime variable, persists in image
- Use `ARG SPARK_VERSION` then `ENV SPARK_HOME`

Security

- Never run as root in production
- Scan with `docker scout` or `trivy`
- Prefer `distroless` base images
- Secrets via `--secret` flag, never `ENV`

Docker Compose: Full Stack

```yaml
version: "3.9"
services:
  spark-master:
    image: bitnami/spark:3.5
    environment: {SPARK_MODE: master}
    ports: ["8080:8080", "7077:7077"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080"]
      interval: 30s
      retries: 5

  spark-worker:
    image: bitnami/spark:3.5
    depends_on:
      spark-master: {condition: service_healthy}
    environment:
      SPARK_MODE: worker
      SPARK_MASTER_URL: spark://spark-master:7077
    deploy: {replicas: 2}

  minio:
    image: minio/minio:latest
```

Container Networking & Volumes

Network drivers

- **bridge** (default): isolated virtual network; DNS by service name
- **host**: no isolation; container shares host stack
- **overlay**: multi-host (Swarm / K8s CNI)
- **none**: no networking (batch jobs)

Named network in Compose:

```
networks:
  bigdata:
    driver: bridge
```

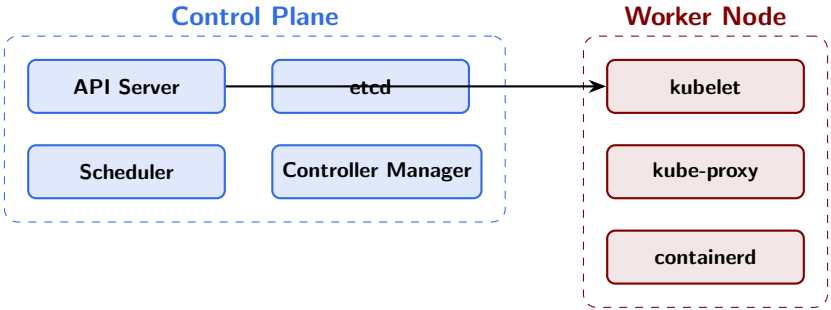
Volume types

- **Named volume**: managed by Docker; survives container removal
- **Bind mount**: maps host path; good for dev hot-reload
- **tmpfs**: in-memory; ephemeral; good for secrets

Bind mount example:

```
docker run \
  -v $(pwd)/data:/app/data \
  my-pipeline:latest
```

Kubernetes Control Plane & Worker Nodes



- **API Server**: single point of truth; all kubectl/REST calls go here
- **etcd**: distributed key-value store for cluster state
- **Scheduler**: assigns pods to nodes based on resources & affinity
- **Controller Manager**: runs reconciliation loops for all object types
- **kubelet**: node agent ensuring containers match PodSpec

Core K8s Objects: Deployment & Service

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pipeline-api
  labels: {app: pipeline-api}
spec:
  replicas: 3
  selector: {matchLabels: {app: pipeline-api}}
  template:
    metadata: {labels: {app: pipeline-api}}
    spec:
      containers:
      - name: api
        image: ghcr.io/iitmz/pipeline-api:v1.2.0
        ports: [{containerPort: 8000}]
        resources:
          requests: {cpu: "250m", memory: "512Mi"}
          limits:   {cpu: "1000m", memory: "1Gi"}
        readinessProbe:
          httpGet: {path: /health, port: 8000}
          initialDelaySeconds: 15
---
```

ConfigMaps, Secrets & ResourceQuota

```yaml
apiVersion: v1
kind: ConfigMap
metadata: {name: pipeline-config, namespace: bigdata}
data:
  SPARK_MASTER: spark://spark-svc:7077
  MINIO_ENDPOINT: http://minio-svc:9000
  LOG_LEVEL: INFO
---
apiVersion: v1
kind: Secret
metadata: {name: pipeline-secrets, namespace: bigdata}
type: Opaque
data:
  MINIO_ACCESS_KEY: bWluaW9hZG1pbg==    # minioadmin
  MINIO_SECRET_KEY: bWluaW9wYXNz        # miniopass
---
apiVersion: v1
kind: ResourceQuota
metadata: {name: bigdata-quota, namespace: bigdata}
spec:
  hard:
    pods: "20"
```

Persistent Storage: PV, PVC, StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: fast}
provisioner: k8s.io/minikube-hostpath
reclaimPolicy: Retain
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: minio-pvc, namespace: bigdata}
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: fast
  resources:
    requests: {storage: 10Gi}
```

- **PV**: cluster-level resource provisioned by admin
- **PVC**: user request bound to a PV
- **StorageClass**: dynamic provisioner (AWS EBS, GCE PD, Azure Disk)
- **AccessModes**: `RWO` (single node), `RWX` (many nodes – NFS/EFS)

StatefulSets for Kafka & Zookeeper

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: kafka, namespace: bigdata}
spec:
  serviceName: kafka-headless
  replicas: 3
  selector: {matchLabels: {app: kafka}}
  template:
    metadata: {labels: {app: kafka}}
    spec:
      containers:
      - name: kafka
        image: bitnami/kafka:3.6
        env:
        - {name: KAFKA_CFG_ZOOKEEPER_CONNECT,
           value: zk-0.zk-svc:2181}
        volumeMounts:
        - {name: kafka-data, mountPath: /bitnami/kafka}
  volumeClaimTemplates:
  - metadata: {name: kafka-data}
    spec:
      accessModes: [ReadWriteOnce]
```

Horizontal Pod Autoscaler

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: pipeline-api-hpa, namespace: bigdata}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: pipeline-api
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
```

Essential kubectl Commands

```
# Apply manifests
kubectl apply -f manifests/              # whole directory
kubectl apply -k overlays/prod/          # kustomize

# Inspect
kubectl get pods -n bigdata -o wide
kubectl describe pod pipeline-api-7f6d-x9k2 -n bigdata
kubectl logs pipeline-api-7f6d-x9k2 -n bigdata --follow

# Debug
kubectl exec -it pipeline-api-7f6d-x9k2 -n bigdata -- /bin/sh
kubectl port-forward svc/pipeline-api-svc 8000:80 -n bigdata

# Rollouts
kubectl rollout status deployment/pipeline-api -n bigdata
kubectl rollout history deployment/pipeline-api -n bigdata
kubectl rollout undo deployment/pipeline-api -n bigdata

# Scaling
kubectl scale deployment pipeline-api --replicas=5 -n bigdata

# Resource usage
```

Helm: Charts, Values, Templates

Chart structure

```
my-pipeline/
  Chart.yaml
  values.yaml
  templates/
    deployment.yaml
    service.yaml
    configmap.yaml
    _helpers.tpl
  charts/
```

Core commands

```
helm repo add bitnami \
  https://charts.bitnami.com/bitnami
helm install kafka bitnami/kafka \
  -f values-kafka.yaml -n bigdata
helm upgrade pipeline ./my-pipeline
    \
  --set image.tag=v1.3.0
helm rollback pipeline 2
helm uninstall kafka -n bigdata
```

Parameterised template snippet

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-api
  labels:
    {{- include "pipeline.labels" . |
        nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: api
        image: "{{ .Values.image.
            repository }}:\
{{ .Values.image.tag }}"
        resources:
          {{- toYaml .Values.resources |
              nindent 10 }}
```

Spark on Kubernetes

```
# Submit directly to K8s API
spark-submit \
  --master k8s://https://<K8S_API_SERVER> \
  --deploy-mode cluster \
  --name spark-pipeline \
  --conf spark.executor.instances=4 \
  --conf spark.executor.cores=2 \
  --conf spark.executor.memory=4g \
  --conf spark.kubernetes.container.image=\
        ghcr.io/iitmz/spark-pipeline:v1.2.0 \
  --conf spark.kubernetes.namespace=bigdata \
  --conf spark.kubernetes.authenticate.driver.serviceAccountName=spark \
  local:///app/pipeline.py
```

- Driver pod created first; executor pods created dynamically
- **spark-on-k8s-operator**: declare `SparkApplication` CRD; operator manages lifecycle
- Dynamic resource allocation: `spark.dynamicAllocation.enabled=true` (Spark 3.4+)
- Airflow **KubernetesExecutor**: each task runs in its own pod

GitHub Actions: Workflow Anatomy

```yaml
# .github/workflows/pipeline.yml
name: Big Data Pipeline CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * 1'          # Weekly Monday 02:00 UTC
  workflow_dispatch:              # Manual trigger

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
```

GitHub Actions: Build & Push Image

```yaml
build-and-push:
  needs: lint-and-test
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
  steps:
  - uses: actions/checkout@v4
  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v3
  - name: Log in to GHCR
    uses: docker/login-action@v3
    with:
      registry: ${{ env.REGISTRY }}
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
  - name: Extract metadata
    id: meta
    uses: docker/metadata-action@v5
    with:
      images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
      tags: |
```

GitHub Actions: Deploy Stage

```yaml
deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment: production

  steps:
  - uses: actions/checkout@v4
  - name: Set up kubectl
    uses: azure/setup-kubectl@v3
  - name: Configure kubeconfig
    run: |
      mkdir -p ~/.kube
      echo "${{ secrets.KUBECONFIG_B64 }}" \
        | base64 -d > ~/.kube/config
  - name: Update image tag
    run: |
      TAG="sha-${{ github.sha }}"
      kubectl set image deployment/pipeline-api \
        api=${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}:${TAG} \
        -n bigdata
  - name: Wait for rollout
```

GitOps Principles & ArgoCD

GitOps core principles

1. **Declarative**: desired state in Git
2. **Versioned**: immutable revisions; full audit trail
3. **Pulled**: agents *pull* from Git (not CI push)
4. **Reconciled**: operators compare actual vs desired

ArgoCD Application manifest

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: {name: pipeline}
spec:
  destination:
    server: https://kubernetes.default.
        svc
    namespace: bigdata
  source:
```

App of Apps pattern

- Root `Application` manages child `Application` objects
- Each team owns a sub-directory
- Full cluster state in one Git repo

Sync phases

1. **PreSync**: DB migrations
2. **Sync**: apply manifests
3. **PostSync**: smoke tests

ArgoCD vs Flux

ArgoCD: UI-rich, multi-cluster. Flux: lightweight, Helm-native. Both implement

Deployment Strategies Compared

Rolling Update (K8s default)

- Replace pods incrementally (`maxSurge`, `maxUnavailable`)
- Zero downtime if health checks are correct
- Old and new versions coexist briefly — beware schema changes

Blue-Green

- Two environments: Blue (live), Green (new)
- Switch traffic via Service selector / Ingress
- Instant rollback: flip selector back

Canary

- Route small % of traffic to new version (e.g., 5%)
- Monitor error rate, latency before full rollout
- Argo Rollouts / Flagger automate progression
- Best for ML model updates with uncertain impact

A/B Testing

- Route by user attribute (header, cookie, region)
- Statistical test before full rollout

Terraform: Provider, Resource, Output

```
# provider.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket = "iitmz-tfstate"
    key    = "bigdata/terraform.tfstate"
    region = "ap-south-1"
  }
}
provider "aws" { region = var.region }

# variables.tf
variable "region"       { default = "ap-south-1" }
variable "cluster_name" { default = "iitmz-bigdata" }

# main.tf
resource "aws_s3_bucket" "data_lake" {
  bucket = "${var.cluster_name}-lake"
}
resource "aws_s3_bucket_versioning" "lake_versioning" {
```

Terraform Workflow & EKS Module

Workflow commands

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform output
terraform destroy
```

State management

- Remote state: S3 + DynamoDB lock
- Never edit `.tfstate` manually
- `terraform import` for existing resources

```
module "eks" {
  source          = "terraform-aws-modules/eks/
      aws"
  version         = "~> 20.0"
  cluster_name    = var.cluster_name
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  eks_managed_node_groups = {
    workers = {
      min_size       = 2
      max_size       = 10
      instance_types = ["m6i.xlarge"]
      capacity_type  = "SPOT"
    }
  }
}
```

Cloud Big Data Services: AWS / GCP / Azure

Capability	AWS	GCP	Azure
Managed Spark	EMR	Dataproc	HDInsight
Serverless ETL	AWS Glue	Dataflow	Synapse Pipelines
Data Warehouse	Redshift	BigQuery	Synapse Analytics
Object Storage	S3	GCS	ADLS Gen2
Managed K8s	EKS	GKE	AKS
ML Platform	SageMaker	Vertex AI	Azure ML
Stream Ingestion	Kinesis	Pub/Sub	Event Hubs

Cost optimisation

- **Spot / Preemptible** instances for Spark workers (up to 90% cheaper)
- **S3 Intelligent-Tiering**: auto-moves cold objects to cheaper storage tiers
- **Auto-scaling** node groups: scale to zero after batch job completes
- Right-size executor memory: monitor GC overhead, not only wall-clock time

Security Best Practices in Kubernetes

RBAC

- Dedicated `ServiceAccount` per workload
- `Role` + `RoleBinding` for namespace scope
- `ClusterRole` only when truly needed
- Principle of least privilege

Network Policies

- Default-deny all ingress/egress
- Explicitly allow required pod-to-pod paths
- Requires Calico or Cilium CNI

Pod Security

- `runAsNonRoot: true`
- `readOnlyRootFilesystem: true`
- `allowPrivilegeEscalation: false`
- Drop all Linux capabilities
- Pod Security Admission (replaces deprecated PSP)

Secrets management

- Sealed Secrets — encrypt before committing to Git
- External Secrets Operator — sync from Vault / AWS SSM

Observability: Metrics, Logs, Traces

Metrics (Prometheus + Grafana)

- ServiceMonitor CRD auto-scrapes pods
- Spark metrics via Prometheus sink
- Kafka JMX exporter
- Alert on: pod crash loop, HPA at max, disk full

Logs (Loki / EFK stack)

- Fluent Bit DaemonSet ships container logs
- Structured JSON logs in application code
- Correlate across Spark jobs via `spark.app.id`

Distributed Tracing (OpenTelemetry)

- Instrument FastAPI with `opentelemetry-instrument`
- Propagate trace context across Kafka messages
- Jaeger or Tempo as trace backend

Data pipeline SLOs

- **Freshness**: data available within N min of event time
- **Completeness**: $\geq 99.9\%$ of records processed
- **Latency**: p99 query ≤ 500 ms

Lab Session Plan (3 Hours)

Part A **Docker** (45 min)

- Write multi-stage Dockerfile for your PySpark pipeline
- Build, inspect layers with `docker history`, run, exec in
- Extend `docker-compose.yml` to include your service

Part B **Kubernetes** (60 min)

- `minikube start --cpus=4 --memory=8192`
- Write Deployment + Service + ConfigMap for FastAPI model server
- `kubectl apply`, port-forward, test endpoint with `curl`
- Scale and observe pod creation

Part C **GitHub Actions** (45 min)

- Fork class repo; create `.github/workflows/pipeline.yml`
- Push: trigger lint $\rightarrow$ test $\rightarrow$ build $\rightarrow$ push GHCR
- Read workflow run logs; fix failures

Part D **Assignment walkthrough** (30 min)

Summary & Key Takeaways

Docker

- Multi-stage builds: keep runtime image lean
- `.dockerignore` + layer ordering = fast CI
- Non-root + HEALTHCHECK + distroless = production-ready

Kubernetes

- Declarative manifests; controllers reconcile state
- StatefulSet for Kafka/ZK; Deployment for stateless
- HPA scales pods; Cluster Autoscaler

CI/CD

- GitHub Actions: trigger → lint → test → build → push → deploy
- GitOps (ArgoCD): Git as single source of truth
- Blue-green: safe cutover; Canary: incremental risk

Cloud & IaC

- Terraform manages cloud infra declaratively
- Spot instances + auto-scaling = cost-efficient Spark
- Observe with metrics, logs, and

References & Further Reading

- Docker Documentation — docs.docker.com
- Kubernetes Documentation — kubernetes.io/docs
- Helm Documentation — helm.sh/docs
- GitHub Actions Docs — docs.github.com/en/actions
- ArgoCD Documentation — argo-cd.readthedocs.io
- Terraform AWS EKS Module — registry.terraform.io
- *Kubernetes in Action* — Lukša Marušič, Manning (2024)
- *Docker Deep Dive* — Nigel Poulton (2023)
- spark-on-k8s-operator — github.com/kubeflow/spark-operator
- Google SRE Book (free) — sre.google/sre-book